

吃豆人迷宫：搜索算法与启发函数设计实验报告

南井然

2026 年秋季学期《人工智能》课程第一次作业

摘要

本实验在 Pacman 框架内实现了 BFS、UCS 与 A* 搜索，并基于反向 Dijkstra 预计算设计了精确启发函数 `yourHeuristic`。实验表明：UCS 与各 A* 变体均能取得最优代价，且启发函数信息量越大，A* 扩展节点数越少。该启发函数在本问题结构下成功达到理论最优搜索效率。项目代码已开源：<https://github.com/sitetampo0724/AU3323>

1 实验任务与问题描述

1.1 任务要求

本次实验目标可概括为：**在扩展节点数尽可能少的前提下，使吃豆人到达终点的累积回报尽可能高**。在本框架中“回报”以路径总代价（Cost）衡量，每走一步扣 1 分，踩入陷阱格扣除大量分数（代价放大 50 倍）。

1.2 吃豆人地图结构与代价模型

地图定义于 `layouts` 目录。图 1 给出两张实验地图：`tinyMaze` (7×7) 与 `mediumMaze` (36×17)。元素含义：P 为吃豆人起点，. 为终点，% 为墙，T 为陷阱。

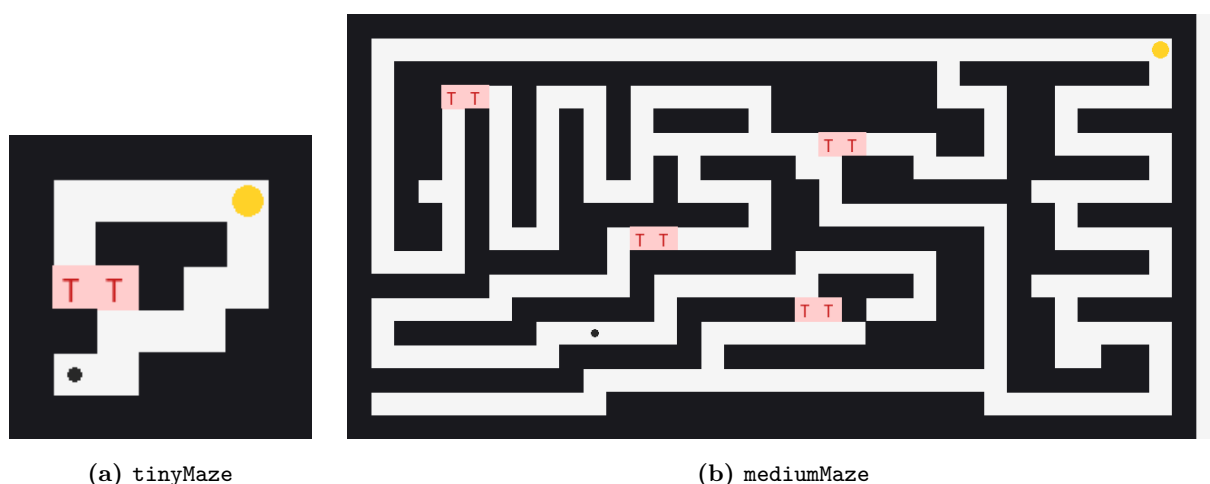


图 1：实验地图：黄色为起点 P，深灰点为终点 .，黑色为墙，红色 T 为陷阱。

代价模型：普通格子行进一步代价为 1，**进入陷阱格时代价放大 `TRAP_MUL=50` 倍**。这一非均匀代价结构是后续算法表现差异的关键，也是启发函数设计的出发点。

2 非启发式搜索算法实现

2.1 统一搜索框架

DFS 的实现给出了清晰的骨架：用 `util.Path(locations, actions, cost)` 表示一个部分分解，用 `fringe` 存放其余部分分解集合，循环弹出、判断目标以及扩展后续部分分解。剩下三个待实现算法与 DFS 的差别仅体现在部分分解弹出优先级规则上。

算法	数据结构	弹出规则	visited 标记时机
DFS	Stack	最深路径先出	路径内判定
BFS	Queue	最浅路径先出	入队时标记
UCS	PriorityQueue	累计代价 $g(n)$ 最小	出队时标记
A*	PriorityQueue	$f(n) = g(n) + h(n)$ 最小	出队时标记

表 1: 四种非启发式/启发式搜索的统一框架对比。

由此可以预测算法在不同情况下的表现：

- **BFS 是代价均匀时的 UCS 特例**：当所有步代价相同时，“步数最少”与“代价最小”等价。然而本实验陷阱代价为 50，BFS 显然将不再代价最优。
- **A* ($h \equiv 0$) 退化为 UCS**：零启发下 $f(n) = g(n)$ ，优先级队列的排序规则与 UCS 完全一致——实验中 `astar+nullHeuristic` 的代价与扩展数应与 UCS 完全相同。

2.2 核心代码

A* 的实现如下（BFS/UCS 与之类似，仅优先级函数不同）：

```
def aStarSearch(problem, heuristic=nullHeuristic):
    start = problem.getStartState()
    if problem.isGoalState(start):
        return []

    fringe = util.PriorityQueue() # ordered by cost + heuristic
    fringe.push(util.Path([start], [], 0), heuristic(start, problem))

    visited = set() # A state is settled once popped with its minimum f-cost
    while not fringe.isEmpty():
        current_path = fringe.pop()
        current_loc = current_path.locations[-1]
        if current_loc in visited:
            continue
        visited.add(current_loc)
        if problem.isGoalState(current_loc):
```

```

    return current_path.actions
else:
    for successor in problem.getSuccessors(current_loc):
        next_loc, next_action, next_cost = successor
        if next_loc not in visited:
            locations = current_path.locations[:]; locations.append(next_loc)
            all_actions = current_path.actions[:]; all_actions.append(next_action)
            new_cost = current_path.cost + next_cost
            path = util.Path(locations, all_actions, new_cost)
            fringe.push(path, new_cost + heuristic(next_loc, problem))
return []

```

3 启发函数设计

3.1 曼哈顿距离

启发函数 $h(n)$ 估计从状态 n 到终点的剩余代价。A* 的最优性要求 h 可采纳，此外最好还能保证函数一致性（满足三角不等式 $h(n) \leq c(n, n') + h(n')$ ）。

在此问题的代价模式下，普通行进代价为 1、陷阱格代价为 50。由于每一步代价至少为 1，真实剩余代价 $\geq$ 步数 $\geq$ 曼哈顿距离，因此曼哈顿距离天然可采纳且一致。然而，它对墙和陷阱所消耗的代价毫不知情。

3.2 利用 Dijkstra 算法设计启发函数

设计分两步：

1. 预计算（仅第一次调用执行）：以终点 `problem.goal` 为源点，在网格上运行一次 Dijkstra 算法，图的边权与后继函数的计费规则完全一致：进入陷阱格代价为 50，普通行进代价为 1。得到 `dist[格子]` = 该格子到终点的真实最优代价。
2. 查询：此后每次调用 $O(1)$ 查表返回 `dist[position]`。

启发函数的设计与实现如下：

```

def yourHeuristic(position, problem, info={}):
    dist = getattr(problem, '_yourHeuristicDist', None)
    if dist is None:
        import heapq
        from pacman.searchAgents import TRAP_MUL # lazy import, avoid circular import
        walls, traps, goal = problem.walls, problem.traps, problem.goal
        dist = {goal: 0}
        pq = [(0, goal)]
        while pq: # reverse Dijkstra from the goal

```

```

d, (x, y) = heapq.heappop(pq)
if d > dist.get((x, y), float('inf')):
    continue
for dx, dy in ((0, 1), (0, -1), (1, 0), (-1, 0)):
    nx, ny = x + dx, y + dy
    if walls[nx][ny]:
        continue
    step = TRAP_MUL if (nx, ny) in traps else 1
    nd = d + step
    if nd < dist.get((nx, ny), float('inf')):
        dist[(nx, ny)] = nd
        heapq.heappush(pq, (nd, (nx, ny)))
problem._yourHeuristicDist = dist # cache on the problem object
return dist.get(position, 0)

```

3.3 理论性质：Dijkstra 算法设计的启发函数

本设计直接取 $h(n) = h^*(n)$ 。启发函数满足可采纳性：因为 $h(n) \leq h^*(n)$ 取等号；一致性也成立，由三角不等式 $h^*(n) \leq c(n, n') + h^*(n')$ ，把 h^* 换成 h 即可得到 $h(n) \leq c(n, n') + h(n')$ 。

问题在于，这样取是不是还有改进余地。设 h' 是任意一个可采纳的启发函数，由可采纳性有 $h'(n) \leq h^*(n)$ 。而本设计的 $h(n) = h^*(n)$ ，于是

$$h'(n) \leq h^*(n) = h(n).$$

这说明在每一个结点上， h 都不小于任何其他可采纳的启发函数。 A^* 的扩展结点数由 $f(n) = g(n) + h(n)$ 控制， h 越接近真实代价， f 越接近真实最优代价 C^* ，被扩展的结点就越少；但 h 一旦超过 h^* ，就不再可采纳， A^* 也就失去最优性保证。所以在可采纳的范围内， h 已经取到了允许的最大值。因此在这个问题上该启发函数难有进一步优化的空间。

从 A^* 的扩展条件看，只有 $f(n) < C^*$ 的结点才会被扩展， $f(n) = C^*$ 的结点位于最优路径平台上，是否弹出取决于打破平局的策略。当 $h = h^*$ 时，

$$f(n) = g(n) + h^*(n)$$

就是从起点经过 n 到目标的真实代价。不在最优路径上的结点 $f(n) > C^*$ ，不会被扩展；在最优路径上的结点 $f(n) = C^*$ ，位于平台上。于是真正被扩展的只有 $f(n) < C^*$ 的结点以及平台上的结点，扩展结点数被压到了理论下界。因此取 h^* 时扩展结点数最少。

需要说明的是，这套论证成立的前提是本问题能够廉价地精确求解子问题，也就是 $h^*(n)$ 本身可以算出来。对于状态空间指数级的问题，比如后续 FoodSearchProblem 中状态包含“剩余豆子集合”，精确的 h^* 本身不可计算，这时就不能再取 $h = h^*$ ，只能退而求其次，设计松弛后的启发函数。启发函数的设计空间也正来自这里：在可计算性和准确度之间找平衡。所以“难有优化空间”是在 h^* 可精确求解的前提下成立的；一旦这个前提不成立，启发函数仍然有大量可以调整的余地。

4 实验结果与分析

4.1 全部运行数据

表 2 汇总 12 组真实运行的结果 (mediumMaze 各组运行的终端输出见图 7, 全部 tinyMaze 运行记录见附录 A)。所有数字均为程序实际输出。

地图	算法 (启发函数)	路径总代价	扩展节点数
mediumMaze	DFS	381	168
	BFS	264	267
	UCS	127	245
	A* (nullHeuristic)	127	245
	A* (曼哈顿距离)	127	234
	A* (yourHeuristic)	127	165
tinyMaze	DFS	108	15
	BFS	8	15
	UCS	8	13
	A* (nullHeuristic)	8	13
	A* (曼哈顿距离)	8	13
	A* (yourHeuristic)	8	8

表 2: 12 组运行结果汇总。加粗为该列最优。

通过表中数据可得到扩展数的严格排序: $\text{yourHeuristic}(165) < \text{Manhattan}(234) < \text{nullHeuristic}(245)$, 与三个启发函数信息量 $h^* \geq h_{\text{Manhattan}} \geq 0$ 的排序严格对应——启发函数越接近真实剩余代价, A* 算法搜索效率越高。

4.2 mediumMaze 运行实录

为便于复核, mediumMaze 上 6 组运行的真实终端输出如下 (图 7; 命令、代价与扩展节点数均可复现, -q --frameTime 0 为关闭动画的静默图形参数; tinyMaze 的运行记录见附录 A)。

```
PS D:\SJTU Classes\人工智能\吃豆人\lab1> python main.py -l mediumMaze -p SearchAgent -a fn=dfs -n 1 -q --frameTime 0
[SearchAgent] using function dfs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 381 in 0.0 seconds
Search nodes expanded: 168
```

图 2: DFS @ mediumMaze: 381 / 168

```
PS D:\SJTU Classes\人工智能\吃豆人\lab1> python main.py -l mediumMaze -p SearchAgent -a fn=bfs -n 1 -q --frameTime 0
[SearchAgent] using function bfs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 264 in 0.0 seconds
Search nodes expanded: 267
```

图 3: BFS @ mediumMaze: 264 / 267

```
PS D:\SJTU Classes\人工智能\吃豆人\lab1> python main.py -l mediumMaze -p SearchAgent -a fn=ucs -n 1 -q --frameTime 0
[SearchAgent] using function ucs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 127 in 0.0 seconds
Search nodes expanded: 245
```

图 4: UCS @ mediumMaze: 127 / 245

```
PS D:\SJTU Classes\人工智能\吃豆人\lab1> python main.py -l mediumMaze -p SearchAgent -a "fn=astar,heuristic=nullHeuristic" -n 1 -q --frameTime 0
[SearchAgent] using function astar and heuristic nullHeuristic
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 127 in 0.0 seconds
Search nodes expanded: 245
```

图 5: A*+nullHeuristic @ mediumMaze: 127 / 245

```
PS D:\SJTU Classes\人工智能\吃豆人\lab1> python main.py -l mediumMaze -p SearchAgent -a "fn=astar,heuristic=manhattanHeuristic2" -n 1 -q --frameTime 0
[SearchAgent] using function astar and heuristic manhattanHeuristic2
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 127 in 0.0 seconds
Search nodes expanded: 234
```

图 6: A*+曼哈顿 @ mediumMaze: 127 / 234

```
PS D:\SJTU Classes\人工智能\吃豆人\lab1> python main.py -l mediumMaze -p SearchAgent -a "fn=astar,heuristic=yourHeuristic" -n 1 -q --frameTime 0
[SearchAgent] using function astar and heuristic yourHeuristic
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 127 in 0.0 seconds
Search nodes expanded: 165
```

图 7: A*+yourHeuristic @ mediumMaze: 127 / 165

4.3 图形界面探索区域对比

图 8 展示各算法在 mediumMaze 上的实际探索区域，直观印证前面的理论分析。

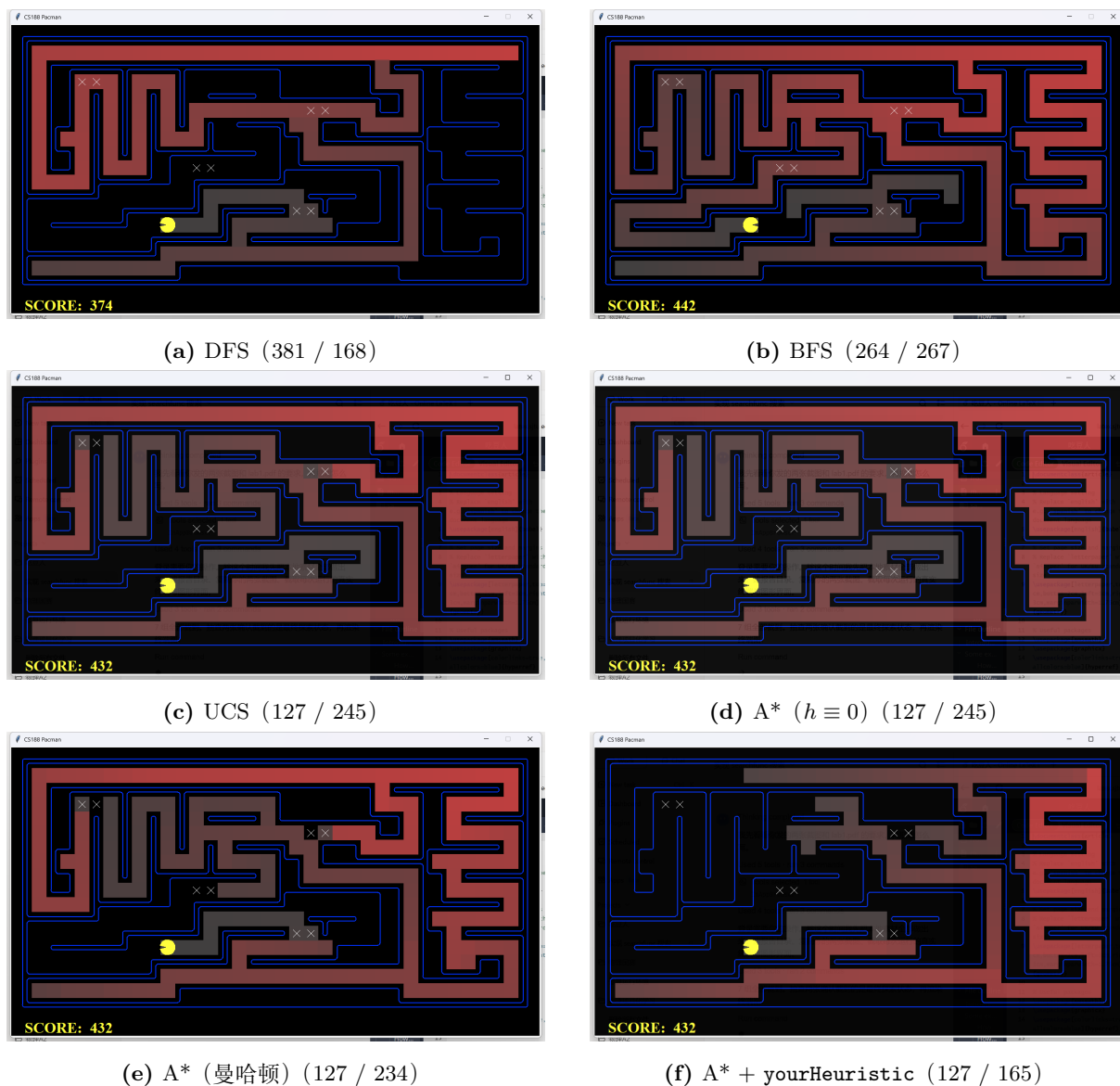


图 8: mediumMaze 上六种算法 - 启发组合找到终点时的真实探索区域。从左到右、从上到下: DFS 深度优先; BFS 逐层铺满; UCS 与零启发 A* 几乎探索全场; 曼哈顿 A* 收缩到目标走廊; yourHeuristic 下 A* 只扩展紧贴最优路径的狭长地带——探索区域随启发函数信息量的增加而单调收缩。

4.4 tinyMaze 验证

在 tinyMaze 上, 所有代价敏感算法 (BFS/UCS/三种 A*) 均得到最优代价 8, 扩展数从 13 降至 8 (图 9): yourHeuristic 的精确代价表让 A* 只扩展最优路径上的结点。

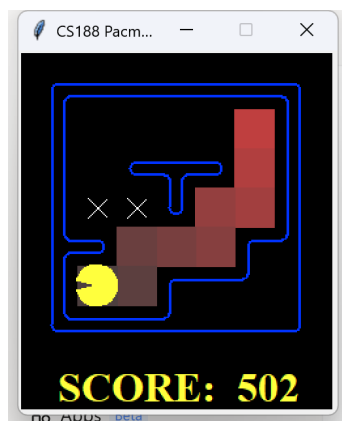


图 9: tinyMaze 上 A* + yourHeuristic: 探索区域即最优路径本身 (8 / 8)。

5 使用 Agent 辅助的心得

本次实验的代码 GPT 6 辅助下完成的。我给了 GPT 一个大致思路，即通过 Dijkstra 算法精确计算启发函数。GPT 肯定了我的想法，并且给出了其为什么是当前问题最优解的证明。我审核过后，没什么问题，就开放权限让他着手写代码。后面我通过命令行手动验证了一下各个算法的总代价和扩展节点数，与预期无误。圆满完成本次任务。

6 结论

本实验完成了 BFS、UCS、A* 的统一框架实现，并设计了以反向 Dijkstra 预计算为核心的精确代价启发函数 `yourHeuristic`。全部代价敏感算法在两张地图上取得最优代价；扩展节点数验证了启发函数信息量的排序：**扩展数 165 (yourHeuristic) < 234 (曼哈顿) < 245 (零启发) = UCS**，`yourHeuristic` 达到可采纳启发函数族的点态上界，是本问题结构下理论最优的 $h(n)$ 设计。

A tinyMaze 运行记录

以下截图为 `tinyMaze` 上 6 组运行的真实命令行界面（命令、代价与扩展节点数均可复现；`-q --frameTime 0` 为关闭动画的静默图形参数；`mediumMaze` 的运行记录见正文第 4.2 节）。

```
PS D:\SJTU Classes\人工智能\吃豆人\lab1> python main.py -l tinyMaze -p SearchAgent -a fn=dfs -n 1 -q --frameTime 0
[SearchAgent] using function dfs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 108 in 0.0 seconds
Search nodes expanded: 15
```

图 10: DFS @ tinyMaze: 108 / 15

```
PS D:\SJTU Classes\人工智能\吃豆人\lab1> python main.py -l tinyMaze -p SearchAgent -a fn=bfs -n 1 -q --frameTime 0
[SearchAgent] using function bfs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 8 in 0.0 seconds
Search nodes expanded: 15
```

图 11: BFS @ tinyMaze: 8 / 15

```
PS D:\SJTU Classes\人工智能\吃豆人\lab1> python main.py -l tinyMaze -p SearchAgent -a fn=ucs -n 1 -q --frameTime 0
[SearchAgent] using function ucs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 8 in 0.0 seconds
Search nodes expanded: 13
```

图 12: UCS @ tinyMaze: 8 / 13

```
PS D:\SJTU Classes\人工智能\吃豆人\lab1> python main.py -l tinyMaze -p SearchAgent -a "fn=astar,heuristic=nullHeuristic" -n 1 -q --frameTime 0
[SearchAgent] using function astar and heuristic nullHeuristic
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 8 in 0.0 seconds
Search nodes expanded: 13
```

图 13: A*+nullHeuristic @ tinyMaze: 8 / 13

```
PS D:\SJTU Classes\人工智能\吃豆人\lab1> python main.py -l tinyMaze -p SearchAgent -a "fn=astar,heuristic=manhattanHeuristic2" -n 1 -q --frameTime 0
[SearchAgent] using function astar and heuristic manhattanHeuristic2
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 8 in 0.0 seconds
Search nodes expanded: 13
```

图 14: A*+曼哈顿 @ tinyMaze: 8 / 13

```
PS D:\SJTU Classes\人工智能\吃豆人\lab1> python main.py -l tinyMaze -p SearchAgent -a "fn=astar,heuristic=yourHeuristic" -n 1 -q --frameTime 0
[SearchAgent] using function astar and heuristic yourHeuristic
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 8 in 0.0 seconds
Search nodes expanded: 8
```

图 15: A*+yourHeuristic @ tinyMaze: 8 / 8